

PRODUCT DOCUMENTATION

Days Between Dates Calculator

User guide & technical documentation — exact day counts between two dates, true calendar breakdowns into years, months and days, business-day counts, an add/subtract-days tool, and a batch mode that reads date pairs straight from pasted or uploaded text.

Creator	**Dr. Asif Ahmad**
Product	Days Between Dates Calculator — version 1.0
Category	Productivity
Published by	Asif SaaS Studio
Date	20 October 2026
Format	Single-file web application (index.html, 19.4 KB)

Private by design: everything runs on the user's device — no uploads, no accounts, no tracking. Exact by design: all counts come from integer UTC day-number arithmetic, so leap years and daylight-saving changes can never skew a result.

Contents

1. What this product does — and who it serves
2. The three tools
3. Counting conventions
4. The calendar breakdown algorithm
5. Leap-year rules
6. Batch mode
7. Worked examples (verified)
8. Validation rules
9. How correctness was verified
10. Adding the product to the Asif SaaS Studio catalogue (Wix CMS)
11. Privacy and security model
12. Product information and credits
13. Appendix — the tested core, verbatim

1. What this product does — and who it serves

"How many days is that, exactly?" sounds like a trivial question until a leap day, a month-end, or a deadline convention changes the answer. Days Between Dates Calculator gives one exact answer and shows its working: the total days, a true calendar breakdown (years, months, days), whole weeks, business days and weekend days, and the weekday name of each date. It serves anyone who plans against a calendar — project managers counting working time to a deadline, HR and payroll staff counting leave and notice periods, teachers and students working out course spans and assignment windows, researchers logging intervals between observations, and anyone counting down to (or back from) an event.

Question	Where the answer lives
How many days between these two dates?	Tool 1 — total days, with an optional inclusive count.
How many working days is that?	Tool 1 — business days (Mon–Fri), with weekend days shown separately.
What is that span in years, months and days?	Tool 1 — the calendar breakdown (Chapter 4).
What date is 90 days from today? 45 days before the deadline?	Tool 2 — add or subtract days from any date.
I have a whole list of start/end pairs in a document.	Tool 3 — batch mode reads the pairs from text and exports CSV (Chapter 6).

2. The three tools

2.1 Tool 1 — Two dates

Pick a start and an end date. The result panel shows, immediately: the total days (the number of midnights between the dates — Chapter 3), an optional include the end date count, the calendar breakdown, whole weeks plus remaining days, business days, weekend days, the total hours, and both dates written out in full with their weekday names. Convenience buttons set the end date to today, swap the two dates, or clear the form. If the end date is earlier than the start date, the app says so and still reports the same span measured backwards — a reversed pair is never silently treated as an error.

2.2 Tool 2 — Add or subtract days

Enter a starting date and a whole number of days, then add or subtract. The result is given in full ("Tuesday, 20 October 2026") together with its ISO form — the quickest way to answer deadline questions such as "90 days after submission" or "30 days before expiry". Month-ends, year-ends and leap Februaries roll over automatically because the arithmetic is done on day numbers, not on calendar fields (Chapter 3).

2.3 Tool 3 — Batch mode

Paste text — or upload a .txt, .csv or .md file — with one date pair per line, in any common format and surrounded by any wording ("Leave from 12/02/2026 until 20/02/2026"). The app finds the first two dates on each line, calculates the span, tabulates the results, and exports them as CSV. Lines it cannot read are flagged, never guessed (Chapter 6).

3. Counting conventions

Every number the app produces follows four stated conventions. They are printed in the app itself, because a day count is only useful if the reader knows exactly what was counted.

Convention	What it means
Midnights between dates	The difference between two dates is the number of midnights crossed from one to the other: 20 Oct → 21 Oct = **1 day**. This is the standard used by calendars and project tools, and it is what date subtraction means everywhere.
Include the end date (+1)	Ticking the box adds exactly 1 to the total, turning the count into an inclusive one — the right convention for hotel-style day counts, age-style counting, and "both endpoints count" deadlines. The result label says the end date was included.
Business days = Mon–Fri	Business days are the Mondays to Fridays inside the same span convention (the start date counts if it is a weekday; the end date counts only when the inclusive option is on, or via the inclusive figure in the examples below).
Public holidays are NOT deducted	Holidays are country-specific — and often region-, sector- and year-specific. Rather than pretend to know the reader's holiday calendar, the app counts Monday–Friday only and says so plainly. If a span contains public holidays, subtract them from the business-day figure yourself.

> **Why UTC day numbers?** Internally, every date is converted to an integer day number (days since the Unix epoch, in UTC) and all arithmetic — differences, additions, weekday lookups — is done on those integers. Local time zones and daylight-saving changes shift clock times, never whole UTC days, so a count can never come out as 29.96 days rounded the wrong way. There is also no backend and no API key involved: the maths is exact integer arithmetic in the page.

4. The calendar breakdown algorithm

A span such as "1 year, 2 months, 5 days" cannot be computed by dividing by 365 or 30 — months have different lengths and leap years intervene. The app computes a true calendar breakdown in three steps:

1. **Whole years.** Take the year difference between the two dates and add that many whole years to the start date (with month-end clamping, below). If the result overshoots the end date, step back one year.
2. **Whole months.** From that anchor, add whole calendar months one count at a time; if adding a month overshoots the end date, step back one month.
3. **Remaining days.** Whatever is left is an exact day difference between the final anchor and the end date.

End-of-month clamping

Adding a month to the 31st of a month that has no 31st lands on that month's last day instead of spilling into the next month: 31 January + 1 month = 29 February in a leap year

(28 in a common year). This clamping is what makes the breakdown come out the way a person would state it:

Span	Breakdown	Why
31 Jan 2024 → 1 Mar 2024	**1 month, 1 day**	31 Jan + 1 month clamps to 29 Feb (2024 is a leap year); 1 day remains to 1 Mar.
29 Feb 2024 → 28 Feb 2025	**Exactly 1 year**	The leap-day anniversary clamps to 28 Feb 2025, leaving 0 months and 0 days.

Because the clamped anchor dates are real calendar dates, the three parts always add back up to the exact total span — a property the automated tests check across leap and common years alike.

5. Leap-year rules

The app uses the standard Gregorian rule, applied in full — including the century corrections that quick calculators often omit:

> **A year is a leap year if it is divisible by 4 — except century years, which are leap years only if divisible by 400.**

Year	Rule applied	Leap?	Consequence
2024	Divisible by 4, not a century year	Yes	28 Feb 2024 → 1 Mar 2024 = **2 days** (29 Feb exists).
2023	Not divisible by 4	No	28 Feb 2023 → 1 Mar 2023 = **1 day**.
1900	Century year, not divisible by 400	**No**	February 1900 had 28 days — the classic case naive "every 4 years" logic gets wrong.
2000	Century year, divisible by 400	**Yes**	February 2000 had 29 days.

Month lengths come from the same rule set (February = 29 days in a leap year, 28 otherwise), and date validation uses it too: 29 February is accepted exactly in the years where it exists.

6. Batch mode

6.1 Recognised formats

Each pattern is searched for anywhere in the line; the first two recognised dates become the pair.

Format	Example	Read as
ISO	`2026-01-05`	5 January 2026 — unambiguous, recommended.
Slash / dash, numeric	`12/02/2026`, `12-02-2026`	12 February 2026 (see the ambiguity rule below).
Day, month name	`5 Jan 2026`	5 January 2026. Full month names and the "Sept" abbreviation also work.
Month name, day	`Jan 5, 2026`	5 January 2026 (US written style).

6.2 The ambiguity rule — stated, never hidden

A numeric date like `05/06/2026` can be read two ways. The app resolves it by a fixed rule, printed in the app and repeated here:

- If the **first** number is above 12, it can only be a day → day/month/year (`13/06/2026` = 13 June).
- If the **second** number is above 12, it can only be a day → month/day/year (`06/13/2026` = 13 June).
- If **both** are 12 or below, the date is read as **day/month/year** — the convention in Pakistan and most of the world (`05/06/2026` = 5 June 2026).

6.3 Nothing is guessed

- **Invalid dates are skipped, never repaired.** A "date" like 31/02/2026 matches the numeric pattern but is not a real calendar date; it fails validation and is ignored, exactly as if it were not there. If that leaves a line with fewer than two valid dates, the line is flagged.
- **Lines without a pair are flagged.** The results table reports "No date pair found (N dates recognised) — line skipped", and the summary counts how many of the lines calculated. A flagged line is information, not an error: it tells you exactly which lines of your document still need attention.
- **Reversed pairs are fine.** If the second date on a line is earlier than the first, the span is reported as a positive count (the same convention as Tool 1).

6.4 CSV export

The Download CSV button exports the successfully calculated lines, one per row:

```
line,start_date,end_date,days_between
1,2026-01-05,2026-03-14,68
2,2026-02-12,2026-02-20,8
3,2026-01-05,2026-02-27,53
```

Dates are normalised to ISO form in the export, so the file drops straight into a spreadsheet or a project tool regardless of the formats the source text used.

7. Worked examples (verified)

Every figure below was reproduced by running the app's shipped calculation core — the same code printed in the Appendix — before this document was written.

7.1 The batch sample (the app's Sample button)

Line	Pair found	Days
Project kickoff 2026-01-05 to 2026-03-14	5 Jan 2026 → 14 Mar 2026	**68**
Leave from 12/02/2026 until 20/02/2026	12 Feb 2026 → 20 Feb 2026 (day/month/year)	**8**
Course: Jan 5, 2026 – Feb 27, 2026	5 Jan 2026 → 27 Feb 2026	**53**
Visa issued (no return date yet)	—	Flagged: no date pair found (0 dates recognised)

7.2 Two-date examples

Span	Result
28 Feb 2024 → 1 Mar 2024	**2 days** (leap year — 29 Feb lies between them).
28 Feb 2023 → 1 Mar 2023	**1 day**.
31 Jan 2024 → 1 Mar 2024	30 days total; breakdown **1 month, 1 day** (clamping — Chapter 4).
29 Feb 2024 → 28 Feb 2025	365 days total; breakdown **exactly 1 year**.
Mon 19 Oct 2026 → Fri 23 Oct 2026	4 days total; **4 business days** — or **5 business days** with "include the end date", because both the Monday start and the Friday end are weekdays and both endpoints then count.
20 Oct 2025 + 365 days (Tool 2)	**Tuesday, 20 October 2026** (2026-10-20).

8. Validation rules

Input	Rule	What the user sees
Start / end date	Both must be real calendar dates (the browser's date picker enforces the shape; the core re-validates year, month and day, including leap Februaries).	"Please choose two valid calendar dates." until both are valid; impossible dates such as 2026-02-30 or 2023-02-29 can never produce a count.
Reversed dates	Not an error.	The same span is reported, with a note that the end date is earlier than the start date.
Day count (Tool 2)	Must be a finite whole number (positive, negative handled by the Add/Subtract buttons).	"Enter a whole number of days." for fractions, text or empty input.
Batch text	Must contain at least one non-empty line; each line needs two recognised, valid dates.	Empty input is prompted for; short lines are flagged per line (Chapter 6.3) rather than failing the whole run.
Batch file upload	.txt, .csv and .md files, read as text on the device.	The file's text replaces the paste box and is calculated immediately.

9. How correctness was verified

The product was verified by **43 automated checks** run against the exact calculation core that ships in the page (Appendix). The checks cover:

- **Core differences** — 1-day spans, same-day = 0, reversed dates, full-year spans of 365, leap-year spans of 366.
- **Leap rules** — February 2024 spans 2 days across the leap day, February 2023 spans 1; **1900 is not a leap year**; 2000 is.
- **Validation** — 2026-02-30, 2023-02-29, month 13 and non-date input all rejected; 2024-02-29 accepted.
- **Add/subtract** — year-end rollover, subtracting back across a leap February, +365 days, +0 days.
- **Breakdowns** — the clamping case (31 Jan → 1 Mar = 1 month, 1 day), a 3 years / 4 months / 5 days span, and the leap-day anniversary = exactly 1 year.
- **Business days** — Monday→Friday = 4 (5 inclusive), a full week = 5, weekend-only spans = 0, and reversed arguments giving the same answer; weekday names verified against a known date (19 Oct 2026 is a Monday).
- **Batch extraction** — ISO pairs (68 days), slash pairs (8 days, read as day/month/year), both month-name styles (53 and 4 days), a line with no pair flagged with 0

dates found, an invalid 31 Feb skipped rather than guessed, ambiguous 05/06 read as 5 June, 06/13 resolved as month/day, and the CSV output format.

43 automated checks — all passed. The figures in Chapter 7 were additionally re-run against the shipped core during the writing of this documentation.

10. Adding the product to the Asif SaaS Studio catalogue (Wix CMS)

1. **Step 1.** Wix dashboard → CMS → **SaasProducts** → Add Item.
2. **Step 2.** Title: **Days Between Dates Calculator**. Category: **Productivity**. Slug: `days-between-dates`.
3. **Step 3.** Summary: "Exact days between two dates — calendar breakdown, business days, add/subtract days, and a batch mode that reads date pairs from pasted or uploaded text. CSV export included." Version: 1.0. Demo: No.
4. **Step 4.** Paste the full contents of `index.html` into the **appHtml** field.
5. **Step 5.** Set **published** on, save, then open the live app page and run the testing checklist from the test report (leap-year spot checks, swap dates, the batch sample with its 68 / 8 / 53-day results).

11. Privacy and security model

- Everything is local — date arithmetic and batch text scanning run in the page; pasted text and uploaded files are never sent anywhere.
- No localStorage, cookies, accounts, analytics or network calls of any kind; the page works fully offline.
- No service workers, permission prompts or parent-window access — sandbox-iframe safe.
- No backend and no API keys are required for any feature of this product.

12. Product information and credits

Field	Detail
Product	Days Between Dates Calculator
Version	1.0 (20 October 2026)
Creator	**Dr. Asif Ahmad**
Publisher	Asif SaaS Studio
Category / slug	Productivity / days-between-dates
Deliverable	index.html — 19,379 bytes (19.4 KB), single file, all CSS and JavaScript inline
Requirements	Any modern browser; no server, no installation, no account
Verification	43 automated checks against the shipped core — all passed (Chapter 9)

Appendix — the tested core, verbatim

The code below is the complete working core of the product exactly as it ships inside `index.html`: the leap-year and month-length rules, date validation, UTC day-number arithmetic, the clamped calendar breakdown, business-day counting, weekday names, and the batch-mode date extraction and CSV writer. Interface wiring is omitted.

```
var MS_DAY = 86400000;
function isLeap(y){ return (y % 4 === 0 && y % 100 !== 0) || y % 400 === 0; }
function daysInMonth(y, m){ return [31, isLeap(y) ? 29 : 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31][m - 1]; }
function validYMD(y, m, d){
  return Number.isInteger(y) && Number.isInteger(m) && Number.isInteger(d) &&
         y >= 1 && y <= 9999 && m >= 1 && m <= 12 && d >= 1 && d <= daysInMonth(y, m);
}
function dayNumber(y, m, d){ return Math.round(Date.UTC(y, m - 1, d) / MS_DAY); }
function ymdFromDayNumber(n){
  var dt = new Date(n * MS_DAY);
  return { y: dt.getUTCFullYear(), m: dt.getUTCMonth() + 1, d: dt.getUTCDate() };
}
function parseISODate(str){
  var m = /^(\d{4})-(\d{1,2})-(\d{1,2})$/.exec(String(str || "").trim());
  if (!m) return null;
  var y = Number(m[1]), mo = Number(m[2]), d = Number(m[3]);
  return validYMD(y, mo, d) ? { y:y, m:mo, d:d } : null;
}
function toISODate(p){
  function pad(n, w){ var s = String(n); while (s.length < w) s = "0" + s; return s; }
  return pad(p.y, 4) + "-" + pad(p.m, 2) + "-" + pad(p.d, 2);
}
function diffDays(a, b){ /* b - a in whole days; both {y,m,d} */
  return dayNumber(b.y, b.m, b.d) - dayNumber(a.y, a.m, a.d);
}
function addDays(p, n){
  return ymdFromDayNumber(dayNumber(p.y, p.m, p.d) + n);
}
/* Calendar breakdown from a to b (a <= b): whole years, months, then days. */
function addMonthsClamped(p, months){
  var total = (p.y * 12 + (p.m - 1)) + months;
  var y = Math.floor(total / 12), m = (total % 12) + 1;
  var d = Math.min(p.d, daysInMonth(y, m));
  return { y:y, m:m, d:d };
}
```

```
function breakdownYMD(a, b){
  var years = b.y - a.y;
  var anchor = addMonthsClamped(a, years * 12);
  if (diffDays(anchor, b) < 0){ years--; anchor = addMonthsClamped(a, years * 12); }
  var months = (b.y - anchor.y) * 12 + (b.m - anchor.m);
  var anchor2 = addMonthsClamped(anchor, months);
  if (diffDays(anchor2, b) < 0){ months--; anchor2 = addMonthsClamped(anchor, months); }
  var days = diffDays(anchor2, b);
  return { years: years, months: months, days: days };
}
/* Business days (Mon–Fri) in [a, b) by default; includeEnd adds b itself. */
function businessDays(a, b, includeEnd){
  var start = dayNumber(a.y, a.m, a.d), end = dayNumber(b.y, b.m, b.d);
  if (end < start){ var t = start; start = end; end = t; }
  var limit = includeEnd ? end + 1 : end, count = 0;
  for (var n = start; n < limit; n++){
    var dow = new Date(n * MS_DAY).getUTCDay();
    if (dow >= 1 && dow <= 5) count++;
  }
  return count;
}
function weekdayName(p){
  return ["Sunday","Monday","Tuesday","Wednesday","Thursday","Friday","Saturday"][new Date(dayNumber(p.y, p.m,
p.d) * MS_DAY).getUTCDay()];
}
/* ---------------- Text extraction for batch mode ---------------- */
var MONTHS = { jan:1, feb:2, mar:3, apr:4, may:5, jun:6, jul:7, aug:8, sep:9, sept:9, oct:10, nov:11, dec:12 };
var DATE_PATTERNS = [
  { re: /\b(\d{4})-(\d{1,2})-(\d{1,2})\b/g, map: function(m){ return { y:+m[1], m:+m[2], d:+m[3] }; } },
  { re: /\b(\d{1,2})[\/\-.](\d{1,2})[\/\-.](\d{4})\b/g, map: function(m){
      var a = +m[1], b = +m[2];
      if (a > 12) return { y:+m[3], m:b, d:a };        /* first must be the day */
      if (b > 12) return { y:+m[3], m:a, d:b };        /* second must be the day -> first is month */
      return { y:+m[3], m:b, d:a };                    /* ambiguous: day/month/year */
  } },
  { re: /\b(\d{1,2})\s+(Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Sept|Oct|Nov|Dec)[a-z]*\s*,?\s*(\d{4})\b/gi, map:
function(m){ return { y:+m[3], m:MONTHS[m[2].toLowerCase().slice(0, 4) === "sept" ? "sept" :
m[2].toLowerCase().slice(0, 3)], d:+m[1] }; } },
  { re: /\b(Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Sept|Oct|Nov|Dec)[a-z]*\s+(\d{1,2})\s*,?\s*(\d{4})\b/gi, map:
function(m){ return { y:+m[3], m:MONTHS[m[1].toLowerCase().slice(0, 4) === "sept" ? "sept" :
m[1].toLowerCase().slice(0, 3)], d:+m[2] }; } }
];
function findDatesInText(text){
  var found = [];
  DATE_PATTERNS.forEach(function(p){
    var m; p.re.lastIndex = 0;
    while ((m = p.re.exec(text)) !== null){
      var pdate = p.map(m);
      if (validYMD(pdate.y, pdate.m, pdate.d)) found.push({ date: pdate, index: m.index, raw: m[0] });
    }
  });
  found.sort(function(x, y){ return x.index - y.index; });
  /* drop overlaps (same start index from two patterns) */
  var out = [], lastIndex = -1;
  found.forEach(function(f){ if (f.index !== lastIndex){ out.push(f); lastIndex = f.index; } });
  return out;
}
function analyzeBatch(text){
  var lines = String(text || "").split(/\r?\n/);
  var results = [];
  lines.forEach(function(line, i){
    if (!line.trim()) return;
    var dates = findDatesInText(line);
    if (dates.length >= 2){
      var d1 = dates[0].date, d2 = dates[1].date;
      results.push({ line: i + 1, label: line.trim(), start: toISODate(d1), end: toISODate(d2), days:
Math.abs(diffDays(d1, d2)), ok: true });
    } else {
      results.push({ line: i + 1, label: line.trim(), ok: false, found: dates.length });
    }
  });
  return results;
}
function batchToCSV(results){
  var lines = ["line,start_date,end_date,days_between"];
  results.forEach(function(r){
    if (r.ok){
```

```
      var label = /[",\n]/.test(r.label) ? '"' + r.label.replace(/"/g, '""') + '"' : r.label;
      lines.push(r.line + "," + r.start + "," + r.end + "," + r.days);
      void label;
    }
  });
  return lines.join("\r\n");
}
```